MemoryVault AI — Complete Team Briefing

Comprehensive Technical Architecture, ML Models, Algorithms, Tech Stack & Terminal Run Guide

---

1. Executive Summary & Project Description

MemoryVault AI is an intelligent, context-aware web application designed to solve a universal daily friction: **misplacing everyday essential items** (such as car keys, wallets, passports, reading glasses, and wireless earbuds).

Instead of relying on random searching or manual Bluetooth hardware tags (AirTags/Tile) which require battery maintenance and prior attachment, MemoryVault AI leverages **Machine Learning (Habit Profiling)** and **Graph Search Routing (A* Pathfinding)** to predict exact locations based on human situational context.

Core Problem Solved: Replaces chaotic, frantic searching with a mathematically optimal, room-by-room search sequence checklist.

Key Innovation: Combines Machine Learning probabilistic classification (Random Forest + Gradient Boosting) with spatial layout graph search (A*) to minimize walking time.

2. End-to-End Operational Workflow

The application operates through a seamless 4-stage pipeline:

- **Stage 1: Conversational Triage (Slot-Filling State Machine)**
  The user interacts with an AI assistant on the left Chat Pane. The system progressively collects context (target item, time of day, routine disruption, emotional state, last activity).
- **Stage 2: Context Feature Extraction & ML Habit Profiling**
  Context inputs are normalized and fed into the trained ML Ensemble Model to calculate room probability distributions across 8 household rooms.
- **Stage 3: Spatial Graph Routing (Modified A* Heuristic)**
  The A* Search Router calculates edge walking distances and prioritizes high-probability locations to produce an expected-search-time-optimized route.
- **Stage 4: Interactive Blueprint & Route Inspector**
  The right pane renders an architectural blueprint floor plan with neon search paths, step sequence badges (#1, #2), distance pills, and an expandable HD modal.

3. Synthetic Dataset & ML Algorithms

3.1 Synthetic Habit Dataset Overview

The ML models are trained on a synthetic dataset of **5,000 contextual habit records** generated via probabilistic distribution sampling. Features include: *item_type*, *time_of_day*, *day_type* (weekday/weekend), *stress_level* (low/medium/high), *has_guests* (boolean), *routine_disrupted* (boolean), and *last_activity* (cooking, working, resting, entering home, rushing).

3.2 Machine Learning Algorithms & Mathematical Role

Algorithm	Function & How It Works	Problem Solved
Random Forest Classifier	Ensemble of 100 decision trees computing non-linear feature splits across contextual variables to output location probabilities.	Prevents overfitting and handles complex non-linear habit interactions.
Gradient Boosting Classifier	Sequential boosting decision tree ensemble that iteratively minimizes residual loss on hard-to-predict contextual edge cases.	Boosts accuracy for high-stress or disrupted routine edge cases.
K-Means & PCA Clustering	K-Means groups contextual vectors into habit clusters; Principal Component Analysis (PCA) projects 8D feature space into 2D scatter coordinates.	Provides visual habit cluster insights in the Evaluator Analytics drawer.

4. Spatial Search Router (A* Pathfinding)

A* Graph Search Heuristic: Traditional shortest-path A* finds the shortest geometric path. MemoryVault AI utilizes a modified cost function that balances *walking distance in meters* against *room item probability*:

$$\text{Cost}(v_{i-1} \to v_i) = \frac{\text{Distance}(v_{i-1}, v_i)}{P(v_i) + \epsilon}$$

What Problem It Solves: Searching room A then room B across the house can waste significant time if a medium-probability room is directly on the way. A* pathfinding computes the single most time-efficient room sequence starting from the main entrance.

5. Technology Stack & Library Breakdown

Component Layer	Technologies & Libraries	Purpose
Backend Server	**Python 3.10+, FastAPI, Uvicorn, Pydantic**	Asynchronous REST API framework for fast request processing and OpenAPI documentation.
Machine Learning	**Scikit-Learn, Joblib, Pandas, NumPy**	Model training, pipeline serialization, metrics calculation, and feature encoding.
Frontend Framework	**Next.js 16 (Turbopack), React 19, TypeScript**	Server-side rendering, component-driven UI architecture, and fast client-side state management.
Styling & Icons	**Vanilla CSS (Tokens), TailwindCSS, Lucide-React**	Custom dark-theme blueprint design system, responsive split-pane, and crisp icons.
Graphics & Blueprint	**SVG (Scalable Vector Graphics), HTML5 Canvas**	High-contrast architectural floor plan map, neon A* paths, Confusion Matrix & ROC curves.

6. Terminal Run & Stop Commands (Step-by-Step)

Team members can follow these exact commands in 2 separate terminal windows to run and test the project locally.

Terminal #1: Backend (FastAPI Python Server)

Exact Target Folder Path: C:\Users\parab\OneDrive\Desktop\Projects\Memory_vault-ai\memoryvault-ai\backend

```
# 1. Navigate to backend folder
cd backend

# 2. Install dependencies (First time only)
pip install -r requirements.txt

# 3. Train ML Models & compute metrics
python -m app.ml.train

# 4. Launch FastAPI Server
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Terminal #2: Frontend (Next.js Application)

Exact Target Folder Path: C:\Users\parab\OneDrive\Desktop\Projects\Memory_vault-ai\memoryvault-ai\frontend

```
# 1. Navigate to frontend folder
cd frontend

# 2. Install npm packages (First time only)
npm install

# 3. Launch Next.js Dev Server
npm run dev
```

Browser Access URL: ■ Open **http://localhost:3000** in your browser.

How to Stop the Project

- **Method 1 (Keyboard):** Press Ctrl + C in both terminal windows.
- **Method 2 (PowerShell Command):** Run the following commands to forcefully kill ports 3000 and 8000:

```
# Stop Frontend on Port 3000
Get-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess -ErrorAction
SilentlyContinue | Stop-Process -Force

# Stop Backend on Port 8000
Get-Process -Id (Get-NetTCPConnection -LocalPort 8000).OwningProcess -ErrorAction
SilentlyContinue | Stop-Process -Force
```

7. Key Presentation Talking Points & Live Demo Guide

- **1. Introduce the Hook (30 sec):** Ask the audience/judges: *'How many times have you wasted 15 minutes searching for your car keys or wallet before leaving home?'*
- **2. Explain MemoryVault AI Solution (45 sec):** Highlight that MemoryVault AI replaces blind panic with ML habit predictions and A* pathfinding.

- **3. Live Demo Flow (2 min):**

a. Open http://localhost:3000.
b. Select **Car Keys** and answer the quick-reply prompts (e.g. Evening, Rushing, High Stress).
c. Show the **Search Route Inspector** blueprint map updated automatically with neon paths, distance markers (meters), and step badges (#1, #2).
d. Click **'Expand Map'** to showcase the HD Fullscreen architectural modal.
e. Click **'Evaluator Analytics'** in the top right to demonstrate model accuracy, confusion matrix, and ROC curves.
f. Click **'Search Another Item'** to demonstrate instant session reset for a second item (e.g. Wallet or Passport).

- **4. Summary & Q&A; Prep (30 sec):** Reiterate that the app uses real spatial graph optimization and self-learning feedback loops when the user clicks 'Found Here'.